

THEORY AND METHODS

AN INTRODUCTION TO NEURAL NETWORKS

P.J. de Jongh: Institute for Maritime Technology, P.O. Box 181,
and Simon's Town, 7995. South Africa.
T. de Wet

Key words: Backpropagation; classification; discriminant analysis; neural networks; non-linear regression; non-parametric regression; pattern recognition.

Summary: Currently an ever increasing number of articles on Neural Networks are appearing in especially the engineering and computer science journals. Neural Networks are applied to a wide variety of problems, but considerable successes have been obtained when applying Neural Networks to classification problems. Statisticians concern themselves with such problems and have, over the years, proposed various techniques for solving classification problems. In this article we will discuss the motivations for, the fundamentals and some applications of Neural Networks. The paper will introduce this exciting new area to statisticians and hopefully will serve to stimulate statistical research into the subject.

1. INTRODUCTION

The human brain is made up of a special kind of cell called the neuron. Billions of these neurons are densely interconnected and form a biological neural network. In an attempt to imitate the workings of biological neural networks, engineers and computer scientists have developed artificial neural networks or so-called connectionist networks. Although these models are inspired by the architecture of the human brain and aspire to imitate human intelligence, they do not exactly mimic the brain's functionality. However, connectionist neural networks have achieved impressive successes in areas

where the human brain, until recently, outperformed even the fastest digital computer. Examples of such areas are speech decoding, text-to-speech conversion, sonar target recognition and character recognition. Sejnowski and Rosenberg (1986) provided one of the most impressive demonstrations of training a neural network in order to learn the mapping from ASCII characters to English phonemes. The first training session resulted in a network making babbling sounds, but after more sessions the network incrementally demonstrated an ever increasing ability of speaking intelligible English.

Most of the successful applications have been in the pattern recognition and classification domains, areas which until recently relied heavily on statistical methods. In the light of this it is surprising that only a handful of statisticians are giving any attention to this new class of models. A wealth of literature on neural networks has already appeared in the journals of diverse fields such as psychology, engineering, computer science, and even operations research, but very little in statistics. It is even more disconcerting when one realises that most neural network models seem to be inherently statistical techniques. (This will be discussed in more detail later.) Therefore we feel it is imperative that statisticians become involved in this subject and it is our hope that this paper will serve to stimulate such research. In the next section we will discuss the motivations for, and the fundamentals of neural network models. Some applications of neural networks to real world data sets will be given in section 3. A statistical perspective on neural networks follows in section 4 and the paper is concluded with some directions for statistical research into the subject.

2. BIOLOGICAL MOTIVATION AND FUNDAMENTALS

The two main reasons for investigating connectionist neural networks are firstly, that they resemble the brain much more closely than conventional computers and secondly, that they are massively parallel. An understanding of their computational properties may lead to a deeper understanding of the

human brain and this will probably allow us to build better neural network models. The parallel architecture of most neural networks can be implemented on parallel hardware having tremendous speed advantages. Therefore, the dream of building intelligent thinking machines is not so distant as many of us might be inclined to believe. In the remainder of this section we will firstly motivate connectionist networks using the analogy of the human brain and then we will discuss the principles of designing and training a special class of connectionist networks, called feedforward neural networks.

As mentioned previously, a human brain consists of billions of neurons densely interconnected by a complex network of synapses, and intelligent behaviour in a human being seems to emerge from interactions involving large numbers of neurons. An example of a biological neuron, the fundamental component of the brain, is shown in figure 1(a). Such a neuron consists of:

dendrites	i.e. neuron branches which connect to other neurons and provide the input (activation or inhibition) to the neuron,
axon	i.e. a long neural fibre that carries the neuron's output to other neurons' dendrites,
synapse	i.e. a junction between the dendrite and axon. This is the connection through which an electro-chemical charge passes from an axon to dendrites,
cell body	the cell body converts input provided by dendrites to an output which is then carried by the axon to the dendrites of other neurons.

The biological neuron continuously receives input from other neurons' axons or from sensing external stimulations which may originate from the

eyes, ears, etc. These input activations are then carried to the cell body. This input activation either excites or inhibits the neuron. When the excitation level rises beyond a certain threshold, the neuron fires and sends an output activation via its axon to the dendrites of other neurons. Each time a neuron fires, the strengths of the junctions (synaptic strengths) are modified. These strengths form the basic memory mechanism of the brain. In summary, each time the brain learns, the synaptic strengths change.

Analogous to the brain, a neural network consists of several processing units (artificial neurons) interconnected in some network architecture. The processing unit corresponds to the biological neuron and an example is depicted in figure 1(b).

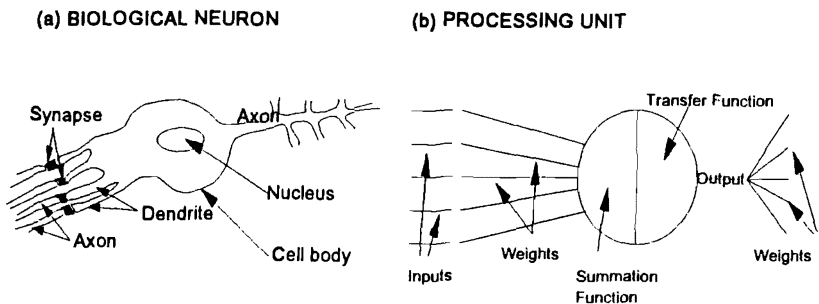


FIGURE 1: Examples of biological and artificial neurons

The particular processing unit consists of four main components:

- | | |
|--------------------|---|
| inputs and outputs | The inputs are the outputs from other neurons or the external input to a given network. (The inputs and outputs correspond to the dendrites and axons, respectively.) |
| weights | These are the quantities determining the strength of the connection between processing units. (The connection weights correspond to the synapses.) |

summation function	This combines the input activations (provided by the inputs and weights) into a single activation received by the neuron.
transfer function	This converts the single activation received by the neuron into an output (sometimes also referred to as the state of the processing unit). The output then becomes the input to other neurons.

The processing unit forms a weighted sum of the inputs from previous layers to which it is connected, adds a threshold value and produces some function of this sum as its output value. In mathematical terms the output o of the processing unit is defined as

$$(1) \quad o = h \left(\sum_{i=1}^k x_i w_i + \theta \right)$$

where the $\{x_i\}$'s are the inputs, the $\{w_i\}$'s the connection weights, θ a threshold value and h some function that is often chosen to be of a sigmoidal form. A popular choice is the *logistic function*

$$(2) \quad h(z) = \left(1 + e^{-z} \right)^{-1}$$

which is a smoothed version of the step function used in the case of neural networks with processing units having discrete states. The processing units are now interconnected in some network architecture of which the *single hidden layer feedforward network* depicted in figure 2 is a very popular choice.

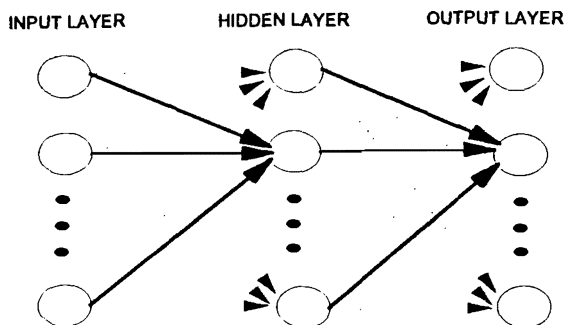


FIGURE 2: A single hidden layer feedforward neural network

It consists of an input layer of processing units, a hidden layer and an output layer. The number of processing units in the input and output layers typically corresponds to the dimension of the input and output vectors respectively, while the user determines the number of processing units in the hidden layer. A rule of thumb is that the number of processing units in the hidden layer should roughly equal the total number of input and output units. The name *feedforward* comes from the fact that the input feeds through the hidden layer to the output layer.

Given a data set consisting of several input and associated output vectors, the neural network is trained to learn the mapping from the input to the output space. When a neural network learns, the weights are modified. Several learning methods, of which the method of *backpropagation* is the most successful and most popular, can be used to adjust the weights.

An outline of the steps followed when *training* a neural network with backpropagation will now be given. For simplicity, we will confine ourselves to single hidden layer feedforward networks. Although networks with more than one hidden layer exist, networks with a single hidden layer are the most popular. (For a more detailed discussion on the method of backpropagation see Werbos 1974 or Rumelhart and McClelland 1986.)

First, one decides on the network topology, viz. the number of processing units in each layer, and the transfer functions of the processing units in each layer. When dealing with function approximation problems, the transfer functions of the input and output units are usually taken to be linear while the transfer functions of the hidden units are taken to be of the sigmoidal form. It is important to note that, in the case of neural network learning via the method of backpropagation, the particular choice of transfer function must be differentiable. The next step is to randomize the weights in the network, usually taken as uniform random numbers between -1 and 1 . The network is then presented with the first example, that is the first input vector and its associated output vector. Given the inputs and the weights, the network output is calculated. This is compared with the associated (desired) output vector and then, typically, the total squared error is calculated. The weights are then adjusted in order to minimize the error at the output layer. The same process is followed when the next example is presented to the network, and training stops when the weights equilibrate (or converge to some optimal value). Hopefully *generalisation* will be achieved, in other words the network will be capable of performing well on data which were not presented to it during training.

The name *backpropagation* comes from the fact that the effect of the error is swept back in that first the weights connecting the hidden and output layer are adjusted and then the weights connecting the hidden layer and input layer (see the Appendix for more details). Backpropagation implements a *gradient descent* approach which minimizes the total squared error at the output layer. The method is particularly important because it was the first method devised by neural network researchers to solve the so-called credit assignment problem, i.e. the problem of adjusting the weights to connections leading into the non-output layers. In section 4, we shall see that this method is known to statisticians as a stochastic approximation to a standard minimization problem (see White 1989a, 1989c).

In this section we have given the reader an introduction to feedforward

networks and explained how the method of backpropagation is used to train them. The interested reader is also referred to the introductory texts by Khanna (1990) and Hecht-Nielsen (1990) as well as the review papers by Masson and Wange (1990) and Ripley (1993).

Remarks

- (i) A neural network can be trained in various ways, of which two are most common. The first is called *supervised training*. Networks trained in this way are presented with a training data set consisting of several input and associated output vectors. (The method of backpropagation is such an example.) Networks trained in this way are called *hetero-associative* networks. The second is called *self-organization*. Networks trained in this way are only presented with input vectors which they then self-organize into different classes. Networks trained in this way are called *auto-associative*.
- (ii) Data sets are usually divided into a training set and a test set. The training set is used to train the network, while the test set is used to evaluate the performance of the network on data which were not used during training. If the network performs well on the test set, generalization has been achieved.
- (iii) Some researchers refer to the network, described in this section, as a multi-layer perceptron. The perceptron is the processing unit characterized by equation (1) and was originally defined by Rosenblatt (1960). His network consisted of processing units with discrete states and was designed to model pattern recognition capabilities of the visual system.
- (iv) The feedforward network is but one of a variety of network architectures and works well for function approximating problems. (Incidentally, note that the weighted sum, equation (1), is not the

only method of combination. Other methods of combination are, for example, the majority function, the logical AND, etc.) Other topologies include, for example, fully connected recurrent networks which are designed to implement spatio-temporal mappings. The latter networks are concerned with mapping a set of time-varying input vectors to a set of time-varying output vectors. See Hecht-Nielsen (1990) for a detailed discussion on different network topologies and learning methods.

- (v) The method of backpropagation can only be used if the transfer functions of all processing units are differentiable. If we are interested in constructing binary networks (i.e. networks consisting of processing units with discrete states), a popular choice for the transfer function is a step or threshold function defined as

$$(3) \quad h(z) = \begin{cases} +1 & \text{if } z > 0 \\ -1 & \text{if } z \leq 0 \end{cases}$$

while a popular learning algorithm is the Madaline Rule II (see Widrow and Lehr 1990 for more details).

- (vi) It is difficult to build neural networks that can be trained effectively by backpropagation, using analog hardware. To overcome this problem the Madaline Rule III was developed which, in principle, produces the same instantaneous error gradient as backpropagation, but is much easier to implement (see Widrow and Lehr 1990 for more details).
- (vii) Once a particular transfer function has been assigned to a processing unit, it is customary to refer to the processing unit by using the name of the particular transfer function. For example, a processing unit having a threshold function as its transfer function, will be referred to as a *threshold unit*. Similarly a processing unit having

the logistic transfer function will be called a *logistic unit*. A *bias unit* is any processing unit having the identity function as its transfer function and which always receives a one as input activation.

3. SOME EXAMPLES

In this section we will apply neural networks to various data sets. This will serve to illustrate the versatility of neural networks, their classification capabilities and the important issues when applying this technology to real world problems.

The exclusive NOR problem

The exclusive NOR problem and the closely related exclusive OR problem (XOR) are probably the most referenced problems in the neural network literature. The reason for this is the controversial book by Minsky and Papert (1969) in which the authors proved that a perceptron (a neural network with no hidden layer) could not implement predicates (binary scalar functions or binary vector variables) such as the XOR logical function. They left the impression that neural network research had to be a dead end, because of its inability to implement essential predicates.

In this section we will show how a neural network can implement the exclusive NOR predicate ($f(+1, +1) = f(-1, -1) = +1$; $f(-1, +1) = f(+1, -1) = -1$) and discuss its classification capabilities in geometrical terms. The exclusive NOR data set is given in table 1 and represented in input space in figure 3(a). The problem is not *linearly separable*, that is no straight line exists which can achieve the separation of the input space corresponding to the desired output space. However, the neural network depicted in figure 3(b) can implement the exclusive NOR logical function. The processing units (PU) in the input layer (PU1, PU2 & PU3) are identity units, while the remaining units (PU4, PU5 & PU6) are threshold units, the threshold function being defined by (3). The threshold units in the hidden layer separate the input space in three decision regions as shown in figure 4.

TABLE 1. The exclusive NOR data

Sample number	Input		Output
	x_1	x_2	y
1	-1	-1	+1
2	-1	+1	-1
3	+1	-1	-1
4	+1	+1	+1

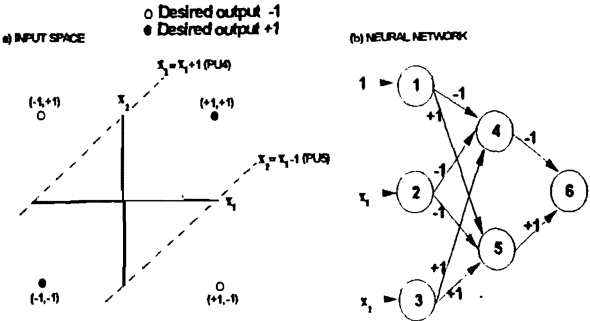


FIGURE 3: Input space of the exclusive NOR problem and neural network implementing it. Note that the numbers on the arcs represent the connection weights

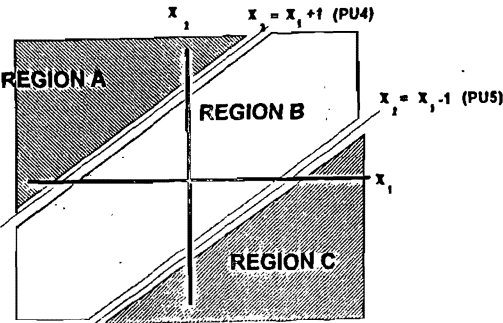


FIGURE 4: Separation of the input space by the neural network depicted in figure 3(b)

For example, region A represents the area $x_2 > x_1 + 1$ (output of PU4 is +1) and $x_2 > x_1 - 1$ (output of PU5 is +1), while region B represents the area $x_2 \leq x_1 + 1$ (output of PU4 is -1) and $x_2 > x_1 - 1$ (output of PU5 is +1). The processing unit in the output layer then groups the regions defined by the hidden layer into new regions yielding the desired outputs. It should be clear that, in general terms, the threshold units in the hidden layer of *single hidden layer feedforward networks* achieve a linear separation of the input space through hyperplanes, while the threshold units in the output layer compute the intersection or union of the linear partitions. In this way arbitrary complex decision regions can be formed. See Lippmann (1987) and Makhoul et al. (1989) for a detailed discussion on the classification capabilities of single hidden layer feedforward networks having threshold units.

Note that replacement of the threshold units (in the hidden layer) with logistic units will result in the separation of the input space being achieved by hypercurves as opposed to hyperplanes.

The IRIS data

The IRIS data set (Fisher, 1936) is composed of 150 four-dimensional feature vectors that measure different characteristics (length and width of petals and sepals) of three species of iris (*I. setosa*, *I. versicolor* and *I. virginica*). This data set has been analyzed on numerous occasions in the statistical literature in order to demonstrate the classification capabilities of various statistical techniques. In this section we compare a single hidden layer feedforward network, trained by the method of backpropagation, with linear discriminant analysis (see e.g. Lachenbruch 1975). We used the software packages Neural Works Professional Plus by NeuralWare and Statgraphics in our analyses. The neural network consisted of 5 linear units (4 input units and 1 bias unit) in the input layer, 10 logistic units in the hidden layer and 3 logistic units in the output layer corresponding to the three species of iris. The choice of the 10 logistic units in the hidden layer is, as is usually the case in neural

network design, somewhat arbitrary, and was guided by the rule of thumb mentioned earlier. The first half of the data set was used to train the neural network and to fit the discriminant functions, while the remainder was used to test the classification performance of the two techniques. The results are summarized in table 2.

Linear discriminant analysis misclassifies one observation in the training data set and four observations in the test data set, while the neural network correctly classifies all the observations in the training set and misclassifies three observations in the test set. (The output unit with the highest output value indicates the choice made for classification.) It is interesting to note that the three observations, misclassified by the neural network, are also misclassified by linear discriminant analysis. We have thus shown that a neural network performs at least as well as linear discriminant analysis on the widely studied iris data. Although this comparison is rather unfair, it serves to illustrate that neural network modelling is a force to be reckoned with.

Remark

The interested reader is referred to the work by Gallinari et al. (1991) who compared linear discriminant analysis to multi-layer perceptrons and proved their equivalence for the linear hetero-associative case. In an earlier study Baldi and Hornik (1989) established the link between principal components analysis and multi-layer perceptrons in the case of auto-association.

TABLE 2. The IRIS data
Linear discriminant analysis

TRAINING SET
PREDICTED GROUP

TEST SET
PREDICTED GROUP

ACTUAL GROUP	<i>I.SETOSA</i>	<i>I.VERSICOLOR</i>	<i>I.VIRGINICA</i>	ACTUAL GROUP	<i>I.SETOSA</i>	<i>I.VERSICOLOR</i>	<i>I.VIRGINICA</i>
<i>I. SETOSA</i>	25			<i>I. SETOSA</i>	25		
<i>I. VERSICOLOR</i>		25		<i>I. VERSICOLOR</i>		24	1
<i>I. VIRGINICA</i>		1	24	<i>I. VIRGINICA</i>		3	22

Neural network

TRAINING SET
PREDICTED GROUP

TEST SET
PREDICTED GROUP

ACTUAL GROUP	<i>I.SETOSA</i>	<i>I.VERSICOLOR</i>	<i>I.VIRGINICA</i>	ACTUAL GROUP	<i>I.SETOSA</i>	<i>I.VERSICOLOR</i>	<i>I.VIRGINICA</i>
<i>I. SETOSA</i>	25			<i>I. SETOSA</i>	25		
<i>I. VERSICOLOR</i>		25		<i>I. VERSICOLOR</i>		24	1
<i>I. VIRGINICA</i>			25	<i>I. VIRGINICA</i>		2	23

4. NEURAL NETWORKS AND STATISTICS

The purpose of the discussion in the preceding sections was to introduce the reader by means of a tutorial to the field of neural networks and to the types of problems to which it can be applied. We have also compared empirically the performance of a popular neural network to that of linear discriminant analysis. Until now, very little has been mentioned regarding neural network performance in a theoretical sense and how and where it fits in with statistics and statistical theory. In this section we view neural networks through the eyes of the statistician and show how statistical theory can help in understanding them better.

Statisticians and mathematicians over the years have concerned themselves, *inter alia*, with finding approximations to mathematical functions or mappings. In this section we will concern ourselves with finding an approximation to the unknown mapping $g: R^k \rightarrow R^m$, which maps an input space to an output space. Here g is sometimes also referred to as the true underlying mapping of the phenomenon to be studied. Most of the approximation approaches revolve around the least squares method and, in that light, can be viewed as variants of least squares regression analysis. Feed-forward neural networks provide an interesting class of models which can be used to approximate the true underlying mapping g . Such networks implement the mapping $f: R^k \times W \rightarrow R^m$, where R^k denotes the input space, W the weight space and R^m the output space. It is important to note that the architecture of the network determines the precise form of f . Network architectures are formed by choosing the number of hidden layers, the number of processing units in each layer and the transfer function of each unit. For each different architecture, we will have a different form of f . In this way a very rich and powerful class of functions f can be generated. For instance, Hornik et al. (1989) have demonstrated that single hidden layer feedforward networks can provide, under certain technical conditions, an accurate approximation to any Borel measurable function $g: R^k \rightarrow R^m$, likely to be encountered, provided that the number of hidden units is large enough. This universal approximation property makes possible the useful application of

feedforward neural networks to a variety of practical problems in the areas of pattern recognition, classification, etc. The above statement is well supported by numerous practical successes achieved to date (see e.g. Sejnowski and Rosenberg 1986 and Hecht-Nielsen 1988, 1990). Failure of such a network must be due to inadequate training, insufficient number of hidden units or the lack of a deterministic relationship between the input and output spaces. However, when confronted with a practical problem, we must decide on the best or optimal network architecture and on the particular learning method to use. Remember that the role of neural network learning is to find suitable values for network weights such that f provides an adequate approximation to g . In the end we want to achieve generalization, i.e. the network must perform well on data which were not presented to the network during training. The above-mentioned practical issues are currently receiving a considerable amount of attention in the neural network literature (see e.g. the papers by Sietsma and Dow 1991 and Hirose et al. 1991). However, most of these papers are empirical in nature and very few papers attempt to resolve the above-mentioned problems rigorously.

Using statistical theory, White (1989a, 1989b) provides a general theoretical framework for addressing these problems. Suppose X and Y are two random variables having joint distribution J . (Note again that according to our previous terminology X denotes an input vector and Y denotes an output vector.) A sample from J (the training set) is then denoted as $(Z_1, \dots, Z_n)$ where $Z_i = (X_i, Y_i)$. We want to approximate g , which we define as $g(X) = E(Y|X)$, with a neural network whose architecture determines the form of f . Network output only depends on the input and weights, so a convenient notation for network output is $f(x, w)$, where x is a realization of the random input vector X and w the vector of weights containing all the weights in a particular network. We now want to choose network weights such that some performance measure, say $\lambda(w)$ is minimized. A natural choice for $\lambda(w)$ is the average performance of the network for a particular loss function, typically assumed to be squared error.

Therefore we have

$$(4) \quad \lambda(\mathbf{w}) = \int (\mathbf{y} - f(\mathbf{x}, \mathbf{w}))^2 d\mathbf{J}(\mathbf{x}, \mathbf{y}).$$

Denote the minimizing value of $\lambda(\mathbf{w})$ by $\mathbf{w}^*$, i.e.

$$(5) \quad \lambda(\mathbf{w}^*) = \min_{\mathbf{w} \in W} \lambda(\mathbf{w}),$$

then $\mathbf{w}^*$ is the vector containing the optimal weights. Because the joint distribution $\mathbf{J}$ is unknown, we cannot obtain $\mathbf{w}^*$ directly and have to estimate it. Several methods to do this exist, of which the method of backpropagation and non-linear least squares regression are the most popular. We will discuss each learning method separately.

Backpropagation

Robbins and Munro (1951) proposed the method of stochastic approximation, which can be used for finding an approximate solution to the problem stated in (5). This is done in the following way. Equation (5) can be written equivalently as solving the set of equations

$$(6) \quad - \int \frac{\partial f(\mathbf{x}, \mathbf{w})}{\partial \mathbf{w}} (\mathbf{y} - f(\mathbf{x}, \mathbf{w})) d\mathbf{J}(\mathbf{x}, \mathbf{y}) = 0.$$

An approximate solution to this equation can be obtained iteratively as follows:

$$(7) \quad \hat{\mathbf{w}}_{\text{BP},i} = \hat{\mathbf{w}}_{\text{BP},i-1} + \alpha \frac{\partial}{\partial \mathbf{w}} f(\mathbf{X}_i, \mathbf{w}) \Big|_{\mathbf{w}=\hat{\mathbf{w}}_{\text{BP},i-1}} (\mathbf{Y}_i - f(\mathbf{X}_i, \hat{\mathbf{w}}_{\text{BP},i-1})),$$

with α a learning rate. Thus, the weight vector on the i -th iteration uses the i -th observation vector. It is not difficult to see that this recursion is nothing but the method of backpropagation which was independently developed by Werbos (1974) and Rumelhart and McClelland (1986). Note that

according to backpropagation we start with a set of random weights $\hat{\mathbf{w}}_{BP,0}$ and then update them iteratively by applying equation (7). It is worth mentioning that, if *all* the observations are fed through the network, their inputs recorded and *then* the weights adjusted, the method of steepest descent is recovered. Neural network learning by the method of backpropagation can be viewed as the problem of estimating the parameters of a misspecified nonlinear regression function (White 1989b). A way of implementing the backpropagation algorithm is given in the Appendix.

Nonlinear regression

Another way of obtaining an approximate solution to (5) is by substituting the empirical distribution J_n for J . Using this approximation, it is possible to compute an approximation to $\lambda(\mathbf{w})$ as

$$(8) \quad \lambda_n(\mathbf{w}) = n^{-1} \sum_{i=1}^n (Y_i - f(X_i, \mathbf{w}))^2$$

which, when minimized, is the well-known least squares regression problem. Denote the solution to minimizing (8) as

$$(9) \quad \lambda_n(\hat{\mathbf{w}}_{NLR}) = \min_{\mathbf{w} \in W} \lambda_n(\mathbf{w}).$$

Therefore, it is clear that neural network output functions correspond precisely to specific families of regression curves. The network inputs are the explanatory or independent variables, and the weights are the regression curve parameters.

From the above it should be clear that both learning methods (backpropagation and non-linear regression) can be viewed as alternative approaches to solving the least squares problem (5). White (1989b) has shown that, under certain technical conditions,

$$\sqrt{n} (\hat{\mathbf{w}}_{BP} - \mathbf{w}^*) \xrightarrow{D} N(0, \mathbf{C}_{BP})$$

(10) and

$$\sqrt{n} (\hat{\mathbf{w}}_{NLR} - \mathbf{w}^*) \xrightarrow{D} N(0, \mathbf{C}_{NLR})$$

where $\xrightarrow{D}$ denotes convergence in distribution. White also noted that $\mathbf{C}_{BP} - \mathbf{C}_{NLR}$ is a positive semi-definite matrix. This implies that non-linear regression is asymptotically efficient with respect to the method of backpropagation. One of the significant consequences of the above results is the fact that the limiting distributions of the two estimators can be used to test hypotheses about $\mathbf{w}^*$. For example, the irrelevant hidden unit hypothesis $H_0: \mathbf{S}\mathbf{w}^* = 0$ can be tested. Here $\mathbf{S}$ is a selection matrix which picks out weights associated with hidden units hypothesized to be irrelevant. The abovementioned hypotheses permit application of the standard methods of statistical inference to questions regarding the precise form of optimal neural network architectures.

Note that, although the method of non-linear regression is asymptotically efficient with respect to the method of backpropagation, the latter has a number of attractive features. The function f needs to be specified in the case of non-linear regression, while the specific form is unimportant in the case of backpropagation. Only the network architecture and the particular transfer functions need to be specified. The latter then typically yield a very complex function f which we need to know if we want to apply model-based non-linear regression techniques. However, in the case of backpropagation, knowledge of the individual, and typically very simple, transfer functions and their derivatives is sufficient.

By now the reader would have realized that *feedforward* neural network models are basically a class of *nonparametric regression* models and therefore possess nonparametric regression capabilities such as kernel and spline

methods. Geman et al. (1992) presented a tutorial on nonparametric inference and its relation to neural networks, and discussed the strengths and weaknesses of neural models. They noted that the substantial problem in neural network modelling is that of identifying the proper network representation, rather than learning *per se*. Geman et al. (1992) also compared feedforward neural networks trained via error backpropagation with nonparametric nearest-neighbour and Parzen-window regression (see e.g. Härdle 1991). The best results obtained for each of these smoothing methods were roughly comparable and no clear winner emerged. In a recent paper, Ripley (1993) claimed that the modelling-based approach traditional in statistics and pattern recognition can at least be as effective, and often more so than neural network models. He illustrated his claim on a practical data set and concluded that in most real-world problems there is other knowledge which can be used to guide modelling and so enable much more convincing generalization than "black-box" learning systems such as neural networks. We as statisticians know that model-based inference suffers from high bias when the underlying model is wrong, opposed to model-free inference which suffers from high variance. The latter is due to the fact that often nonparametric models (such as neural networks) are over-parameterized so that a large number of parameters need to be estimated. Also nonparametric methods are generally slow to converge in the sense that a large number of training samples are required to obtain acceptable performance. It is well known that nonparametric schemes generally perform well when interpolating, but when extrapolating they yield unpredictable results. The above generally holds true for neural networks as well.

Based on the above discussion, the reader is likely to be somewhat confused about the virtues of neural networks. If they are just a class of nonparametric regression models with their associated problems, why all the interest in them? Much of the excitement amongst engineers and computer scientists stems from their dream to build intelligent machines which can learn by example and through learning can articulate the rules and heuristics of complex problems. The realization of this dream is still in the distant

future, but already neuro-computers are being built and already spectacular successes, although limited, have been achieved on difficult unsolved problems. The neural network approach provides a quick and easy way to try out different architectures on almost any kind of problem. It has been said that neural networks will provide a sub-optimal answer to most problems. However, compared to human problem-solving expertise computer time is very cheap. In practice a good, but non-optimal solution is often necessitated by time and money constraints. Therefore we believe it is important that statisticians become involved in neural network research. We have a lot to offer and can potentially gain much in return.

Remark

Geman et al. (1992) also addressed the issue of the choice of the size of the network for a particular problem. In a single hidden layer feedforward network this boils down to deciding on the number of units in the hidden layer, since the problem structure (dimension of the input and output vectors) determines the number of units in the input and output layers. They stated that too small a network (not enough hidden units) is likely to be biased since the repertoire of available functions spanned by $f(\mathbf{x}, \mathbf{w})$ over allowable weights, will be quite limited. If the true regression $E(Y|X)$ is poorly approximated by this class then the bias could be substantial. On the other hand, if the neural network model is over-parameterised via a large number of hidden units and associated weights, then the bias will be reduced, but then there is the danger of a significant variance contribution to the mean squared error. They illustrated this empirically by incrementally increasing the number of hidden units in a single hidden layer feedforward network, each time fitting it to a particular data set and showed that the variance increased with the number of hidden units. The search for an appropriate technique for determining network complexity has received some attention to date. This is an area where the methods developed by statisticians can prove helpful. For instance, White (1989a) has suggested using the

method of cross-validation in order to determine the network complexity appropriate for a training set of a given size.

5. CONCLUDING REMARKS

Neural network methods provide a new and powerful way of solving a variety of problems in pattern recognition, classification, forecasting and other related areas. Although it originated from fields outside statistics, statisticians should seriously consider using and studying them. Inherently, neural networks are close to statistical methods and many practical problems, which concern statisticians, can be tackled with neural networks. Neural network research is still at the beginning of the statistical learning curve. This is evident from the non-standardized use of terminology and the lack of a rigorous (statistical) theory. It thus provides a rich area for academic research. For instance, research into finding asymptotically efficient, computationally efficient and robust learning methods is necessary. Another possible research area is to make use of the theory of statistical inference in order to derive methods for finding optimal network architectures and efficient methods of network pruning. It is also necessary to assess the advantages of using neural networks in practice. For these more empirical studies, it is necessary to compare neural networks with existing statistical methods. To conclude, we believe that neural networks offer challenging new research possibilities and hope that statisticians will take up this challenge.

6. ACKNOWLEDGEMENTS

The authors would like to thank the referees for comments which substantially improved the presentation of the paper.

7. REFERENCES

- BALDI, P. & HORNIK, K. (1989). Neural networks and principal components analysis: Learning from examples without local minima. *Neural networks*, vol. 2, pp. 53-58.

- FISHER, R.A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, vol. 7, pp. 179-188.
- GALLINARI, P., THIRIA, S., BADRAN, F. & FOGELMAN-SOULIE, F. (1991). On the relations between discriminant analysis and multilayer perceptrons. *Neural networks*, vol. 4, pp. 349-360.
- GEMAN, S., BIENENSTOCK, E. & DOURSAT, R. (1992). Neural networks and the bias/variance dilemma. *Neural computation*, vol. 4, pp. 1-58.
- HÄRDLE, W. (1991). *Smoothing techniques with implementation in S*. Springer Series in Statistics. Springer-Verlag, New York.
- HECHT-NIELSEN, R. (1988). Applications of counterpropagation networks. *Neural networks*, vol. 1, pp. 131-139.
- HECHT-NIELSEN, R. (1990). *Neurocomputing*. Addison-Wesley.
- HIROSE, Y., YAMASHITA, K. & SHIMPEI, H. (1991). Backpropagation algorithm which varies the number of hidden units. *Neural networks*, vol. 4, pp. 61-66.
- HORNIK, K., STINCHCOMBE, M. & WHITE, H. (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, vol. 2, pp. 359-366.
- KHANNA, T. (1990). *Foundations of neural networks*. Addison-Wesley.
- LACHENBRUCH, P.A. (1975). *Discriminant analysis*. Hafner Press, Collier Macmillan publishers, London.
- LIPPMANN, R.P. (1987). An introduction to computing with neural networks. *IEEE Transactions on Acoustics, Speech and Signal Processing*, pp. 4-22.
- MAKHOUL, J., SCHWARTZ, R. & EL-JAROUDI, A. (1989). Classification capabilities of two-layer neural nets. *IEEE proceedings, ICASSP*, pp. 635-638.
- MASSON, E., & WANGE, Y-J. (1990). Introduction to computation and learning in artificial neural networks. *European Journal of Operational Research*, vol. 47, pp. 1-28.
- MINSKY, M. & PAPERT, S. (1969). *Perceptrons*. Cambridge, MA: MIT Press.

- RIPLEY, B.D. (1993). *Statistical aspects of neural networks*. Proceedings of the invited lectures for SemStat, Sandbjerg, Denmark (25-30 April 1992). Chapman & Hall.
- ROBBINS, H. & MUNRO, S. (1951). A stochastic approximation method. *Annals of Mathematical Statistics*, vol. 22, pp. 400-407.
- ROSENBLATT, F. (1960). *On the re-inforcement procedures in simple perceptrons*. Cornell Aeronautical Laboratory, Report VG-1196-G-4, New York.
- RUMELHART, D.E. & MCCLELLAND, J.L. (1986). *Parallel distributed processing: Explorations in the microstructures of cognition*. Cambridge, MIT Press.
- SEJNOWSKI, T.J. & ROSENBERG, C.R. (1986). *NETalk: A parallel network that learns to read aloud*. JHU/EECS-86/01, School of Electrical Engineering and Computer Science, Johns Hopkins University.
- SIETSMA, J. & DOW, R.J.F. (1991). Creating artificial neural networks that generalize. *Neural networks*, vol. 4, pp. 67-79.
- WERBOS, P.J. (1974). *Beyond regression: New tools for prediction and analysis in the behavioural sciences*. Unpublished Ph.D. thesis, Applied Mathematics, Harvard University.
- WHITE, H. (1989a). Learning in artificial neural networks: A statistical perspective. *Neural computation* 1, pp. 425-464.
- WHITE, H. (1989b). Some asymptotic results for learning in single hidden layer feedforward network models. *J. Am. Statis. Assoc.*, vol. 84, no. 408, pp. 1003-1013.
- WHITE, H. (1989c). Neural network learning and statistics. *AI Expert*, December 1989, pp. 48-52.
- WIDROW, B. & LEHR, M.A. (1990). 30 years of adaptive neural networks: Perceptron, madaline and backpropagation. *Proceedings of the IEEE*, vol. 78, no. 9, pp. 1415-1442.

APPENDIX

The method of backpropagation

The recursion defined in equation (7) provides a general framework for updating the weights by backpropagation. However, in practice and as was mentioned in section 2, first the weights of the connections leading into the output layer are adjusted and then the weights of the connections leading into the hidden layer. The adjustments are usually made after each training instance (when the network is presented with a particular input vector and its associated output vector). The weights of the connections leading into the j -th output unit are updated by the equation

$$(A1) \quad \mathbf{w}_{ij}^{\text{new}} = \mathbf{w}_{ij}^{\text{old}} + \alpha(\mathbf{y}_j - \mathbf{o}_j)h'(\mathbf{net}_j),$$

where $\mathbf{w}_{ij}$ represents the weight of the connection from the i -th unit in the hidden layer into the j -th output unit, α the learning rate, $\mathbf{y}_j$ the desired output of the j -th unit, $\mathbf{o}_j$ the output of the j -th unit, $h(\cdot)$ the transfer function of the j -th unit and $\mathbf{net}_j$ the input activation received by the j -th unit. Unlike the output units, the hidden units receive no direct feedback from training patterns and the real power of the backpropagation algorithm comes from its ability to cater for this fact. The weight of the connections leading into the j -th unit of the hidden layer is updated by the equation

$$(A2) \quad \mathbf{w}_{ij}^{\text{new}} = \mathbf{w}_{ij}^{\text{old}} + \alpha h'(\mathbf{net}_j) \sum_k \delta_k \mathbf{w}_{jk},$$

where $\mathbf{w}_{ij}$ represents the weight of the connection from the i -th unit in the input layer into the j -th unit in the hidden layer, α the learning rate, $h(\cdot)$ the transfer function of the j -th unit, $\mathbf{net}_j$ the input activation received by the j -th unit, $\mathbf{w}_{jk}$ the weight of the connection between the j -th unit and the k -th output unit and δ_k the amount with which the weight of that

connection was updated by (A1), i.e.

$$(A3) \quad \delta_k = (y_k - o_k)h'(net_k).$$

Note that the method of backpropagation as described by equations (A1), (A2) and (A3) easily generalizes to feedforward neural networks with more than one hidden layer. Also note that the algorithm described here is but one of a variety of ways of implementing the method of backpropagation. See Rumelhart and McClelland (1986), Widrow and Lehr (1990) and Ripley (1993) for more details.